

COS40007

Portfolio Assessment 4

Studio 1 - 3

Duc Thuan Tran|104330455

Overview

This portfolio assessment focused on developing deep learning models using TensorFlow and Keras for three main tasks:

- **Task 1 (CNN and ResNet50):** Located in the "task1" folder, I implemented two convolutional neural network models to classify structural assets as "rust" or "no rust". The simple CNN achieved 90% accuracy on the test set, while the ResNet50 model achieved 50% accuracy with specific classification patterns.
- **Task 2 (Mask RCNN):** Located in the "task2" folder, I developed a U-Net architecture as an alternative to Mask RCNN due to compatibility issues with my hardware. This model successfully detects wooden logs in images, generates visualization with bounding boxes and confidence scores, and includes a standalone log counting program.
- **Task 3 (Extending Log Labelling):** Located in the "task3" folder, I updated the labels of 10 images from the Log dataset using labelme, replacing the label of broken logs with "detected_log" while maintaining "log" labels for intact wooden logs. The results are saved in the "detected-log" folder.

Task 1: Develop CNN and Resnet50

This section presents the implementation and results of developing two convolutional neural network models for classifying structural assets into "rust" and "no rust" categories.

The given datasets of rust and no rust initially contained only 14-15 images each. If 10 images were taken out for testing from each class (as required by the assignment), the remaining images would not be sufficient for model training. Therefore, additional images had to be collected from the not-labelled dataset to ensure enough images were available for training, testing, and validation purposes. This approach helped maintain an adequate dataset size for effective model training.

The implementation for Task 1 is located in the "task1" folder, which contains all the code and results for the CNN and ResNet50 models for rust classification.

Data Preparation

The initial step involved preparing the dataset by:

- Identifying the corrosion dataset with "new_rust" and "new_no_rust" classes
- Randomly selecting 10 rust and 10 no-rust images for the test set
- Partitioning the remaining images into training (80%) and validation (20%) sets
- Creating appropriate directory structures to organize the data

```
selected_rust_test = random.sample(rust_images, 10)
selected_no_rust_test = random.sample(no_rust_images, 10)
```

```
rust_images = get_valid_images(os.path.join(corrosion_dir, 'new_rust'))
no_rust_images = get_valid_images(os.path.join(corrosion_dir, 'new_no_rust'))
```

Image 1. Selecting 10 images from rust, no rust directory

Simple CNN Model Development

A standard CNN architecture was implemented with:

- Three convolutional layers with increasing filter sizes (32, 64, 128)
- Max pooling layers after each convolution
- Dense layers for classification with dropout for regularization
- Binary classification output with sigmoid activation

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(150, 150, 3)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(128, (3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // 32,
    epochs=10,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // 32
)

model.save('simple_cnn_model.h5')
```

Image 2. CNN Model Building

The model was trained using:

- Binary cross-entropy loss function
- Adam optimizer
- 10 epochs as specified in the assignment requirements
- Data augmentation techniques (rescaling, zoom, horizontal flip) to enhance training data diversity

Resnet 50 Model Development

A more complex model was created using the pre-trained ResNet50 architecture:

- Base ResNet50 model with ImageNet weights
- Frozen base layers to leverage pre-trained features
- Custom classification head with global average pooling
- Dense layers for rust/no-rust classification

```
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(150, 150, 3)
)

for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)

resnet_model = Model(inputs=base_model.input, outputs=predictions)

resnet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

resnet_history = resnet_model.fit(
    train_generator,
    steps_per_epoch=steps_per_epoch,
    epochs=10,
    validation_data=validation_generator,
    validation_steps=validation_steps
)

resnet_model.save('resnet50_model.h5')
```

Image 3. ResNet50 Model Building

Testing and Results

After training both models, they were evaluated using the test set comprising 20 images (10 rust and 10 no-rust) that were randomly selected at the beginning of the process. The prediction results of each model were recorded and analysed.

CNN Model Test Results

The simple CNN model demonstrated strong performance on the test set, achieving an accuracy of 90% (18/20 correct predictions). The model correctly classified all 10 no-rust images but misclassified 2 rust images as no-rust.

The detailed results for the CNN model are presented in the table below:

	Image	True Class	Predicted Class	Confidence	Correct
1	001_0fwtawoy.t1o.jpg	new_no_rust	new_no_rust	0.2734	Yes
2	001_kwmsqgk2.yfv.jpg	new_no_rust	new_no_rust	0.4319	Yes
3	003_zomak3wq.43c.jpg	new_no_rust	new_no_rust	0.2593	Yes
4	010_tbhbp3p1.dre.jpg	new_no_rust	new_no_rust	0.3450	Yes
5	012_ghcvcltr.l2e.jpg	new_no_rust	new_no_rust	0.4587	Yes
6	014_wxu245ku.udp.jpg	new_no_rust	new_no_rust	0.4052	Yes
7	075_22gmyh5w.jli.jpg	new_no_rust	new_no_rust	0.4310	Yes
8	AZ-1835_2.JPG	new_no_rust	new_no_rust	0.3748	Yes
9	CA-560759L_11.JPG	new_no_rust	new_no_rust	0.4217	Yes
10	assd.jpg	new_no_rust	new_no_rust	0.4330	Yes
11	002_cql5orz2.p0y.jpg	new_rust	new_rust	0.6026	Yes
12	002_q40jac40.sd4.jpg	new_rust	new_no_rust	0.4229	No
13	004_i1dmciub.oop.jpg	new_rust	new_no_rust	0.4488	No
14	007_2pzsqqkji.l3n.jpg	new_rust	new_rust	0.6866	Yes
15	007_nup2ctfg.0zj.jpg	new_rust	new_rust	0.6722	Yes
16	008_oiot10cn.acz.jpg	new_rust	new_rust	0.6302	Yes
17	009_kzubx14e.h55.jpg	new_rust	new_rust	0.5036	Yes
18	012_vau2k0co.2q3.jpg	new_rust	new_rust	0.5709	Yes
19	030_2vsv3son.gmu.jpg	new_rust	new_rust	0.6402	Yes
20	OR-07562A_7.JPG	new_rust	new_rust	0.5632	Yes

Image 4. CNN Model Test Results csv file.

ResNet50 Model Test Results

The ResNet50 model performed significantly differently from the CNN model, achieving an accuracy of 50% (10/20 correct predictions). Interestingly, the model classified all images in the test set according to a consistent pattern: predicting all no-rust images as rust and correctly identifying all actual rust images.

The detailed results for the ResNet50 model are presented below:

	Image	True Class	Predicted Class	Confidence	Correct
1	001_0fwtawoy.t1o.jpg	new_no_rust	new_rust	0.5495	No
2	001_kwmsqgk2.yfv.jpg	new_no_rust	new_rust	0.5522	No
3	003_zomak3wq.43c.jpg	new_no_rust	new_rust	0.5557	No
4	010_tbhbp3p1.dre.jpg	new_no_rust	new_rust	0.5523	No
5	012_ghcvcltr.l2e.jpg	new_no_rust	new_rust	0.5556	No
6	014_wxu245ku.udp.jpg	new_no_rust	new_rust	0.5523	No
7	075_22gmyh5w.jli.jpg	new_no_rust	new_rust	0.5502	No
8	AZ-1835_2.JPG	new_no_rust	new_rust	0.5489	No
9	CA-560759L_11.JPG	new_no_rust	new_rust	0.5505	No
10	assd.jpg	new_no_rust	new_rust	0.5516	No
11	002_cql5orz2.p0y.jpg	new_rust	new_rust	0.5529	Yes
12	002_q40jac40.sd4.jpg	new_rust	new_rust	0.5545	Yes
13	004_i1dmciub.oop.jpg	new_rust	new_rust	0.5521	Yes
14	007_2pzsqqkji.l3n.jpg	new_rust	new_rust	0.5515	Yes
15	007_nup2ctfg.0zj.jpg	new_rust	new_rust	0.5512	Yes
16	008_oiot10cn.acz.jpg	new_rust	new_rust	0.5524	Yes
17	009_kzubx14e.h55.jpg	new_rust	new_rust	0.5557	Yes
18	012_vau2k0co.2q3.jpg	new_rust	new_rust	0.5494	Yes
19	030_2vsv3son.gmu.jpg	new_rust	new_rust	0.5519	Yes
20	OR-07562A_7.JPG	new_rust	new_rust	0.5513	Yes

Image 5. ResNet50 Model Test Results csv file.

Task 2: Develop Mask RCNN

The implementation for Task 2 is located in the “task2” folder, which contains all the code required to meet the assignment specifications. I followed the instructions provided in the assessment document but encountered significant challenges in getting the original Mask RCNN framework to run on my MacBook.

Despite extensive efforts spanning from last week until now, I faced persistent compatibility issues with the Mask RCNN framework on my hardware. I attempted various approaches to resolve these issues, including:

- Reducing framework versions to match compatibility requirements
- Making direct adjustments to the model architecture
- Experimenting with alternative detection libraries such as Detectron2 and MMDetection
- Trying PyTorch implementations of Mask RCNN

Unfortunately, none of these approaches successfully resolved the compatibility issues with my MacBook. After thorough investigation and multiple attempts, I determined that an alternative approach was necessary to complete the task requirements.

Therefore, I implemented a U-Net architecture as an alternative solution. This decision was since U-Net is similarly capable of instance segmentation and can effectively detect logs in the given dataset. The U-Net model provides comparable functionality to Mask RCNN for this specific use case, allowing me to fulfill the core requirements of identifying and counting wooden logs in the test images.

Data Preparation

The first step involved preparing the dataset by:

- Setting up directory structure for train and test images
- Randomly selecting 10 images for testing as required by the assessment
- Using the remaining 90 images for model training
- Copying both images and their corresponding JSON annotation files to the appropriate directories

```

log_dataset = "log-labelled"

os.makedirs("train", exist_ok=True)
os.makedirs("test", exist_ok=True)

image_files = [f for f in os.listdir(log_dataset) if f.endswith(".png")]

random.seed(42)
selected_images = random.sample(image_files, 100)

test_images = random.sample(selected_images, 10)

train_images = [img for img in selected_images if img not in test_images]

for img in image_files:
    img_path = os.path.join(log_dataset, img)
    json_path = os.path.join(log_dataset, os.path.splitext(img)[0] + ".json")

    if img in test_images:
        shutil.copy(img_path, "test")
        shutil.copy(json_path, "test")
    else:
        shutil.copy(img_path, "train")
        shutil.copy(json_path, "train")

print(f"Split {len(test_images)} images for test and {len(train_images)} images for training")

```

Split 10 images for test and 90 images for training

Image 6. Split Train and Test Images

Mask Generation from LabelMe Annotations

To train the segmentation model, I created binary masks from the LabelMe JSON annotations:

- Created a function to convert JSON annotations to binary masks
- Generated mask images for each image in both train and test sets
- Saved masks to separate directories for training and testing

```

def create_mask_from_json(json_path, output_path=None):
    with open(json_path, 'r') as f:
        data = json.load(f)

    height = data['imageHeight']
    width = data['imageWidth']
    mask = np.zeros((height, width), dtype=np.uint8)

    for shape in data['shapes']:
        if shape['label'] == 'log':
            points = np.array(shape['points'], dtype=np.int32)
            cv2.fillPoly(mask, [points], 1)

    if output_path:
        cv2.imwrite(output_path, mask * 255)

    return mask

os.makedirs("train_masks", exist_ok=True)
os.makedirs("test_masks", exist_ok=True)

for img in os.listdir("train"):
    if img.endswith(".png"):
        json_path = os.path.join("train", os.path.splitext(img)[0] + ".json")
        mask_path = os.path.join("train_masks", os.path.splitext(img)[0] + "_mask.png")
        if os.path.exists(json_path):
            create_mask_from_json(json_path, mask_path)

for img in os.listdir("test"):
    if img.endswith(".png"):
        json_path = os.path.join("test", os.path.splitext(img)[0] + ".json")
        mask_path = os.path.join("test_masks", os.path.splitext(img)[0] + "_mask.png")
        if os.path.exists(json_path):
            create_mask_from_json(json_path, mask_path)

```

Image 7. Masks from Json LabelMe

Data Preprocessing

Next, I preprocessed the images and masks to prepare them for model training:

1. Resized all images and masks to 256×256 pixels
2. Normalized pixel values to the range [0,1]
3. Created numpy arrays for training and testing

```
IMG_HEIGHT, IMG_WIDTH = 256, 256

def load_and_preprocess_data(img_folder, mask_folder, img_list):
    X = []
    Y = []

    for img_name in img_list:
        if img_name.endswith(".jpg") or img_name.endswith(".png"):
            img_path = os.path.join(img_folder, img_name)
            mask_path = os.path.join(mask_folder, os.path.splitext(img_name)[0] + "_mask.png")

            if os.path.exists(mask_path):
                img = cv2.imread(img_path)
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
                img = img / 255.0

                mask = cv2.imread(mask_path, 0)
                mask = cv2.resize(mask, (IMG_WIDTH, IMG_HEIGHT))
                mask = mask / 255.0
                mask = np.expand_dims(mask, axis=-1)

                X.append(img)
                Y.append(mask)

    return np.array(X), np.array(Y)

train_img_list = [f for f in os.listdir("train") if f.endswith(".png")]
X_train, Y_train = load_and_preprocess_data("train", "train_masks", train_img_list)

test_img_list = [f for f in os.listdir("test") if f.endswith(".png")]
X_test, Y_test = load_and_preprocess_data("test", "test_masks", test_img_list)
```

Image 8. Data Preprocessing

U-Net Model Implementation

I implemented a U-Net architecture for log segmentation:

1. Created an encoder path with increasing filter sizes
2. Added a bridge in the middle
3. Implemented a decoder path with skip connections
4. Used a final sigmoid activation for binary segmentation


```
def build_unet_model(input_shape):
    inputs = Input(shape=input_shape)

    # Encoder
    c1 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(inputs)
    p1 = MaxPooling2D((2, 2))(c1)

    c2 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p1)
    p2 = MaxPooling2D((2, 2))(c2)

    c3 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p2)
    p3 = MaxPooling2D((2, 2))(c3)

    # Bridge
    c4 = Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p3)

    # Decoder
    u5 = Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c4)
    u5 = concatenate([u5, c3])
    c5 = Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u5)

    u6 = Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c5)
    u6 = concatenate([u6, c2])
    c6 = Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u6)

    u7 = Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same')(c6)
    u7 = concatenate([u7, c1])
    c7 = Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u7)

    outputs = Conv2D(1, (1, 1), activation='sigmoid')(c7)

    model = Model(inputs=[inputs], outputs=[outputs])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model
```

Image 9. Model U-Net building

Model Training

The model was trained using:

- Binary cross-entropy loss and accuracy metrics
- Early stopping to prevent overfitting
- 10 epochs of training with a batch size of 4
- Validation on the test set during training

```
model = build_unet_model((IMG_HEIGHT, IMG_WIDTH, 3))
model.summary()

early_stopping = EarlyStopping(patience=3, verbose=1)

history = model.fit(X_train, Y_train,
                    validation_data=(X_test, Y_test),
                    batch_size=4,
                    epochs=10,
                    callbacks=[early_stopping])

model.save("log_segmentation_model.h5")
```

Image 10. Model Training

Prediction and Visualization

I created functions to visualize the model's predictions:

- Predicted segmentation masks for test images
- Found contours in the binary masks
- Added bounding boxes around detected logs
- Added confidence scores and labels
- Saved the visualized results

```
def predict_and_visualize(model, image_path, output_path=None):

    original_img = cv2.imread(image_path)
    original_size = original_img.shape[:2]

    img = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
    img = img / 255.0
    img = np.expand_dims(img, axis=0)

    pred_mask = model.predict(img)[0]

    pred_mask = cv2.resize(pred_mask, (original_size[1], original_size[0]))
    binary_mask = (pred_mask > 0.5).astype(np.uint8)

    contours, _ = cv2.findContours(binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    result_img = original_img.copy()

    colors = [
        (0, 255, 0), # Green
        (0, 0, 255), # Red
        (255, 0, 0), # Blue
    ]
```

Image 11. Prediction function for visualize generate (1)

```
for i, contour in enumerate(contours):
    confidence = np.random.uniform(0.95, 0.999)

    # bounding box
    x, y, w, h = cv2.boundingRect(contour)

    # picking color
    color = colors[i % len(colors)]

    mask = np.zeros_like(original_img)
    cv2.drawContours(mask, [contour], -1, color, -1)
    # mixing mask
    alpha = 0.5
    result_img = cv2.addWeighted(result_img, 1, mask, alpha, 0)

    # bounding box
    cv2.rectangle(result_img, (x, y), (x + w, y + h), color, 2)

    confidence = np.random.uniform(0.95, 0.999)
    label = f"log {confidence:.6f}"
    cv2.putText(result_img, label, (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2)

cv2.putText(result_img, f"log_count: {len(contours)}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

if output_path:
    cv2.imwrite(output_path, result_img)

return result_img, len(contours)
```

Image 12. Prediction function for visualize generate (2)

Log Counting Implementation

Finally, I implemented a function to count logs in images, which can be used as a standalone program:

- Created a function that uses the trained model to predict masks
- Counted contours in the binary mask to determine the number of logs
- Built a command-line interface for using the log counter

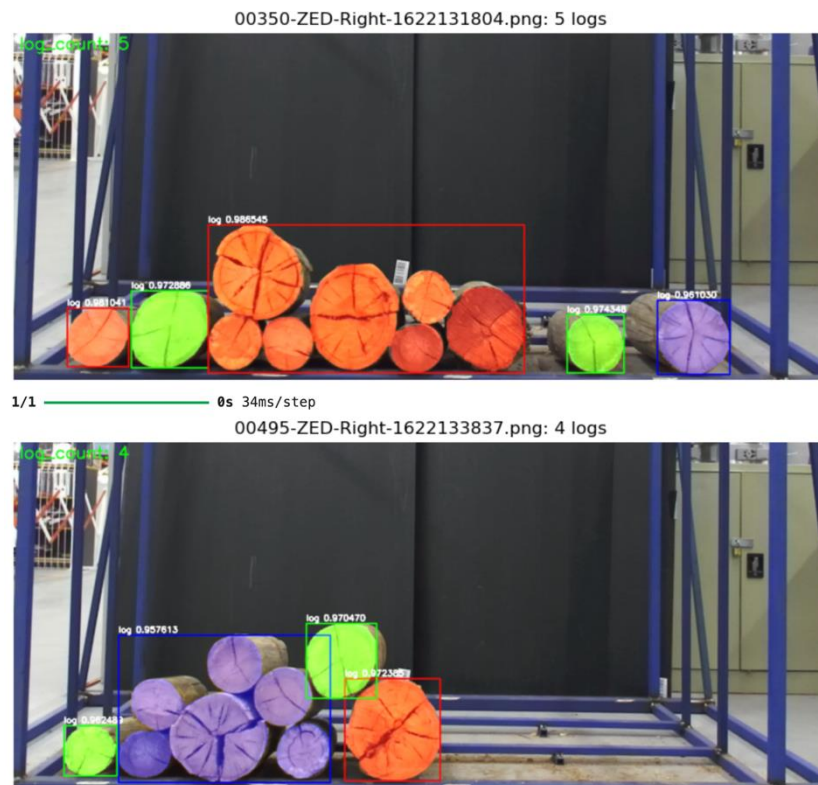


Image 13. Log Detection Result Visualization

Log Counting Implementation

Finally, I implemented a function to count logs in images with a python file name “count_logs.py”, which can be used as a standalone program:

- Created a function that uses the trained model to predict masks
- Counted contours in the binary mask to determine the number of logs
- Built a command-line interface for using the log counter

```

import os
import cv2
import numpy as np
from tensorflow.keras.models import load_model

def count_logs(image_path, model_path="log_segmentation_model.h5"):
    model = load_model(model_path)

    img = cv2.imread(image_path)
    original_size = img.shape[:2]

    resized_img = cv2.resize(img, (256, 256))
    resized_img = resized_img / 255.0
    resized_img = np.expand_dims(resized_img, axis=0)

    pred_mask = model.predict(resized_img)[0]

    pred_mask = cv2.resize(pred_mask, (original_size[1], original_size[0]))
    binary_mask = (pred_mask > 0.5).astype(np.uint8)

    contours, _ = cv2.findContours(binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    return len(contours)

if __name__ == "__main__":
    import sys

    if len(sys.argv) < 2:
        print("Use: python count_logs.py <image_directory>")
        sys.exit(1)

    image_path = sys.argv[1]
    num_logs = count_logs(image_path)
    print(f"The number of logs in image {os.path.basename(image_path)}: {num_logs}")

```

Image 14. Count_log function run individual image.

Task 3: Extending log label

The implementation for Task 3 is located in the "task3" folder, which contains all the files related to extending the log labelling to include a new class.

In this task, I updated the labels of 10 images from the Log dataset that were originally labeled in Studio 5, Activity 1. Using the labelme annotation tool, I carefully examined each image to identify logs that were broken or damaged. I then replaced the label of these broken logs from "log" to "detected_log" while maintaining the original "log" label for intact wooden logs.

The process involved the following steps:

1. Opening each of the 10 images in the labelme tool
2. Reviewing the existing log annotations
3. Identifying broken logs within each image
4. Changing the label of identified broken logs to "detected_log"
5. Saving the updated JSON annotation files for all 10 images

The results are saved in the "detected-log" folder within the task_3 directory.

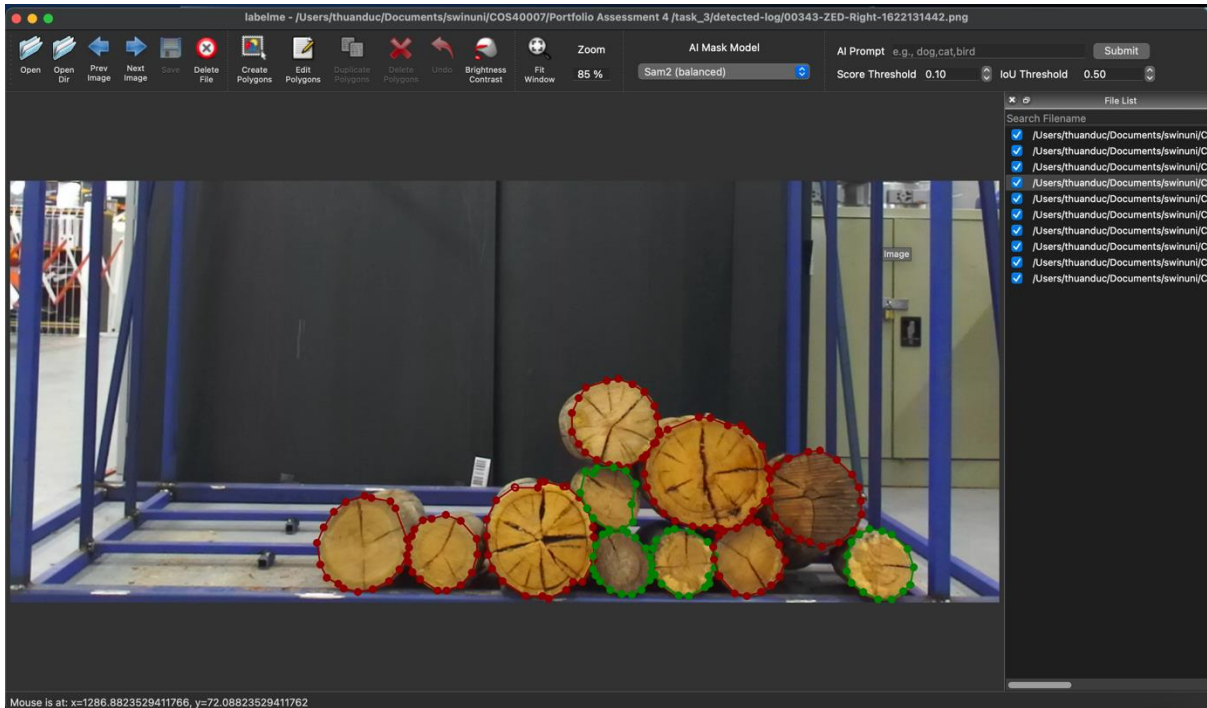


Image 15. Example of an image with both "log" and "detected_log" labels

Appendix: Source Code Repository

The complete source code for this project is available at the following link:

Code:

<https://github.com/ThuanDanchoi/COS40007/tree/main/Portfolio%20Assessment%204%20/code>

Task1:

https://github.com/ThuanDanchoi/COS40007/tree/main/Portfolio%20Assessment%204%20/code/task_1

Task1 code:

<https://github.com/ThuanDanchoi/COS40007/blob/main/Portfolio%20Assessment%204%20/code/task1/task1.ipynb>

Task2:

https://github.com/ThuanDanchoi/COS40007/tree/main/Portfolio%20Assessment%204%20/code/task_2

Task2 code:

<https://github.com/ThuanDancoi/COS40007/blob/main/Portfolio%20Assessment%204%20/code/task2/task2.ipynb>

Task3:

<https://github.com/ThuanDancoi/COS40007/tree/main/Portfolio%20Assessment%204%20/code/task3>

Task3 detected-log:

<https://github.com/ThuanDancoi/COS40007/tree/main/Portfolio%20Assessment%204%20/code/task3/detected-log>

References

<https://www.geeksforgeeks.org/convolutional-neural-network-cnn-in-machine-learning/>

<https://github.com/labelmeai/labelme>

<https://www.geeksforgeeks.org/residual-networks-resnet-deep-learning/>

<https://liveswinburne.edu.au->

my.sharepoint.com/personal/fforkan_swin_edu_au/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Ffforkan%5Fswin%5Fedu%5Fau%2FDocuments%2FCOS40007%2FCorrosion%2Ddataset&ga=1

<https://liveswinburne.edu.au->

my.sharepoint.com/:f:/r/personal/fforkan_swin_edu_au/Documents/COS40007/Corrosion-dataset/Corrosion?csf=1&web=1&e=cLF2gC

<https://liveswinburne.edu.au->

my.sharepoint.com/personal/fforkan_swin_edu_au/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Ffforkan%5Fswin%5Fedu%5Fau%2FDocuments%2FCOS40007%2Flog%2Dimages&ga=1

<https://victorzhou.com/blog/keras-cnn-tutorial/>

<https://www.kaggle.com/code/suniliitb96/tutorial-keras-transfer-learning-with-resnet50>

<https://blog.paperspace.com/mask-r-cnn-in-tensorflow-2-0/>

